

TF-FaultNet in Plain Language

What we built, why it works, and what to say about it

For: the project team

One-line version: we point a microphone at a machine, and a neural network tells us which of 12 things is wrong with it.

1. The problem

Factories and cars are full of spinning parts — belts, pulleys, gears, bearings. When one starts wearing out, catching it early saves a lot of money. Catching it late destroys the machine.

The normal way to catch it is a **vibration sensor bolted onto the machine**. That works, but it's expensive, and you have to physically open up and modify every machine you want to monitor.

Our idea: **just listen to it**. A microphone costs almost nothing, touches nothing, and can be moved anywhere. The question is whether sound alone carries enough information.

It does. That's the whole project.

2. What makes it hard

Three things get in the way:

Faults sound completely different from each other. A slipping belt makes a continuous scraping noise that runs through the whole recording. A gear with a missing tooth makes one sharp *click* every rotation and is silent in between. A model that's good at spotting one is usually bad at the other.

Machines hum constantly. Shaft rotation, motor noise, 50/60 Hz from the mains. That hum is present in every single recording — healthy or broken — so it's pure distraction.

It's easy to fool yourself. If your test data accidentally contains clips from the same recording session as your training data, your model can score 99% by recognising the room, not the fault. A lot of published work has this problem.

3. How the system works

Step 1 — Turn sound into a picture

Raw audio is just a wiggly line, 44,100 numbers per second. Hard to learn from.

So we convert each 1-second clip into a **spectrogram** — a picture where the horizontal axis is time, the vertical axis is pitch, and brightness is how much energy is there. A belt slip shows up as a bright horizontal smear. A broken tooth shows up as thin bright vertical stripes.

Once it's a picture, we can use image-recognition techniques on it.

The one clever bit: most projects make these pictures once, save them as files, and train on the files. We compute them *inside* the model, on the GPU, every time. That means no storage cost, and the model can learn all the way back to the raw audio instead of stopping at a frozen image.

Step 2 — Look at the picture

TF-FaultNet is a fairly standard image classifier (a ResNet) with two additions that exist specifically because of the problems in section 2:

Addition	What it does	Which problem it solves
SE attention	Learns which frequency bands matter and turns the rest down	Kills the constant machine hum
Dual pooling	Records both the <i>average</i> energy and the <i>loudest peak</i>	Catches smeared faults AND spiky faults

Dual pooling is worth understanding properly. If you only take the average, a single millisecond click gets diluted into nothing — you'd miss every tooth-loss fault. If you only take the peak, a continuous scrape has no peak to find — you'd miss every belt fault. So we take both and hand the classifier both numbers.

Step 3 — Prove we're not cheating

Four tests, in the order you'd run them:

1. **Can it memorise 24 clips?** If not, something is broken in the code. It got 100%.
2. **Shuffle the labels and retrain.** Accuracy should collapse to random guessing. It fell to 8.84%, and random for 12 classes is 8.33%. So the model isn't picking up some artifact of the recording setup.
3. **Does validation loss track training loss?** Yes, gap of 0.0115. If it had ballooned, we'd be memorising.
4. **Train on early recordings, test on later ones.** This is the honest one. **97.16%.**

4. The results

Model	Accuracy
TF-FaultNet (ours)	99.77% $\pm$ 0.21%
WaveformCNN1D	99.67% $\pm$ 0.24%
AudioBiGRU	99.30% $\pm$ 0.33%
XGBoost	96.46% $\pm$ 1.04%

12 classes, 2,148 clips, 179 per class, five-fold cross-validation.

Now read that table honestly. The gap between us and WaveformCNN1D is 0.1 percentage points — about two clips out of 2,148 — and the error bars overlap completely. We cannot claim our model is more accurate than that baseline. What the table actually shows is that **three different deep models all max out this dataset**, and only the simple 24-feature approach falls behind.

That's fine. It just means we sell the architecture on different grounds: it's interpretable (you can look at the spectrogram and the attention weights and see *why*), it maps onto the physics of the faults, and it doesn't need offline preprocessing.

5. Which number to quote

Quote 97.16%, not 99.77%.

The 99.77% comes from a random split, where clips from one recording session can land in both training and test. The 97.16% comes from holding out entire later sessions — genuinely unheard recordings. That 2.6-point gap is roughly the size you'd expect from session-level leakage, which is exactly why the second number is the defensible one.

Say it first, before anyone asks. It reads as rigour. If a reviewer has to dig it out of you, it reads as something you were hiding.

6. Questions to be ready for

"Isn't 99.77% suspiciously high?" Yes, and it's because the task is easier than a real factory: controlled testbench, balanced classes, clean recordings. Say so. Then pivot: the point isn't the ceiling, it's that a contactless mic reaches accuracy people currently need bolted-on accelerometers for.

"Why is your model better than the 1D baseline?" On accuracy, it isn't — the difference is noise. It's better on interpretability, on physics justification, and on not needing cached

spectrogram files.

"Did you check for data leakage?" Yes — chronological session holdout, 97.16%. Also the label-permutation test.

"Where are your ablations?" We don't have them, and this is our real weakness. SE attention and dual pooling are justified by argument, not by measurement. Removing each one and re-running the five folds is the single most valuable thing left to do.

7. Five things to remember

1. Microphone instead of a bolted-on sensor. That's the pitch.
2. Sound becomes a picture, and the picture is computed inside the model, not saved to disk.
3. SE attention removes the hum; dual pooling catches both smeared and spiky faults.
4. Lead with 97.16% (unseen sessions), not 99.77%.
5. Our biggest gap is the missing ablations. Own it before someone else points at it.